

Entrega LAB2

1. Introducció

Hem d'implementar un sistema de bloqueig (lock) distribuït per gestionar l'exclusió mútua entre diversos processos (workers) sense utilitzar un servidor central ni memòria compartida. El sistema es basa en pas de missatges en Erlang.

Hem partit d'una implementació bàsica (**lock1**) que patia de problemes de deadlock (bloqueig mutu) i hem evolucionat el sistema a través de dues versions millorades:

- **lock2**: Soluciona el deadlock mitjançant prioritats estàtiques basades en l'identificador del node.
- **lock3**: Millora l'equitat (fairness) utilitzant Relotges Lògics de Lamport per atendre les peticions en ordre causal.

A més, s'ha modificat l'arquitectura per permetre l'execució realment distribuïda en diferents nodes d'Erlang.

2. Implementació i Modificacions del Codi

2.1. Adaptació per a l'entorn distribuït (muty.erl)

El mòdul original estava dissenyat per instanciar tots els processos dins del mateix node local. Per complir amb els requisits de l'experiment distribuït, s'ha implementat una sobrecàrrega de la funció start que accepta una llista de nodes remots.

Canvis principals: S'ha substituït la creació local de processos per l'ús de crides a procediment remot (**rpc**). En lloc d'utilitzar spawn o register directament, utilitzem rpc:call per forçar que la creació del procés (spawn) i el seu registre es realitzin físicament en el node de destí. Això garanteix que cada parella lock-worker s'executi en el seu propi espai de memòria i màquina virtual, comunicant-se realment a través de la xarxa.

Codi destacat:

```
None
start(Lock, Sleep, Work, [N1, N2, N3, N4]) ->
    % Arrancamos los LOCKS en remoto
    % Usamos un rpc:call anidado. El de dentro crea el proceso ALLI. Y el de
    fuera registra ese proceso ALLI.
    rpc:call(N1, erlang, register, [11, rpc:call(N1, Lock, start, [1])]),
    rpc:call(N2, erlang, register, [12, rpc:call(N2, Lock, start, [2])]),
    rpc:call(N3, erlang, register, [13, rpc:call(N3, Lock, start, [3])]),
    rpc:call(N4, erlang, register, [14, rpc:call(N4, Lock, start, [4])]),
```

```

% Conectamos los Peers
% Enviamos la tupla {Nombre, Nodo} para que sepan donde enviar el
mensaje
{11, N1} ! {peers, [{12, N2}, {13, N3}, {14, N4}]},
{12, N2} ! {peers, [{11, N1}, {13, N3}, {14, N4}]},
{13, N3} ! {peers, [{11, N1}, {12, N2}, {14, N4}]},
{14, N4} ! {peers, [{11, N1}, {12, N2}, {13, N3}]},

% Arrancamos los WORKERS en remoto
% Igual que antes, forzamos que el worker nazca ALLI
rpc:call(N1, erlang, register, [w1, rpc:call(N1, worker, start, ["John",
11, Sleep, Work])]),
rpc:call(N2, erlang, register, [w2, rpc:call(N2, worker, start,
["Ringo", 12, Sleep, Work])]),
rpc:call(N3, erlang, register, [w3, rpc:call(N3, worker, start, ["Paul",
13, Sleep, Work])]),
rpc:call(N4, erlang, register, [w4, rpc:call(N4, worker, start,
["George", 14, Sleep, Work])]),
ok.

```

2.2. Solució al Deadlock (lock2.erl)

La implementació inicial (lock1) patia d'interbloqueig (deadlock) quan múltiples nodes sol·licitaven l'accés a la secció crítica de manera concurrent. Per resoldre aquest problema, a lock2 s'ha introduït un mecanisme de resolució de conflictes basat en **prioritat estàtica**. Aquesta prioritat ve determinada per l'identificador únic del node, establint arbitràriament que un **ID menor** implica una prioritat major.

Lògica implementada: S'ha modificat el comportament dins de l'estat d'espera (wait). Quan el procés està esperant per aconseguir el bloqueig i rep una petició externa, avalua la prioritat del sol·licitant respecte a la seva:

- Si el node extern té **més prioritat** (ID < MyId): Es cedeix el pas immediatament enviant un missatge ok. El nostre node continua esperant, però ha evitat el bloqueig circular permetent que el node prioritari avanci.
- Si el node extern té **menys prioritat** (ID > MyId): Es posposa la resposta. La petició s'afegeix a la llista d'espera (Waiting) i serà atesa només quan nosaltres alliberem el bloqueig, mantenint el comportament original.

A nivell d'estructura, ha estat necessari propagar la variable MyId a través de totes les funcions d'estat (open, wait, held) per garantir que la prioritat local estigui accessible en el moment de la comparació.

Codi destacat:

```

None
wait(Nodes, Master, Refs, Waiting, TakeRef, MyId) ->
    receive
        {request, From, Ref, Rid} ->
            % Si el que pide (Rid) tiene mayor prioridad (valor menor) que
            yo (MyId),
            % le deixo pasar enviando {ok, Ref}, aunque yo este esperando.
            if
                Rid < MyId ->
                    From ! {ok, Ref},
                    wait(Nodes, Master, Refs, Waiting, TakeRef, MyId);
                true ->
                    % Si yo tengo prioridad, lo deixo esperando
                    wait(Nodes, Master, Refs, [{From, Ref}|Waiting],
TakeRef, MyId)
            end;
        {ok, Ref} ->
            NewRefs = lists:delete(Ref, Refs),
            wait(Nodes, Master, NewRefs, Waiting, TakeRef, MyId);
    release ->
        ok(Waiting),
        open(Nodes, MyId)
    end.

```

2.3. Implementació de Relotges de Lamport (lock3.erl)

Tot i que la versió lock2 resol el problema de l'interbloqueig, introdueix un problema d'inanició (starvation) o falta de justícia: els nodes amb ID baix acaparen el recurs. Per garantir un accés equitatiu (fairness), a lock3 s'ha substituït la prioritat estàtica per una **ordenació temporal** basada en Relotges Lògics de **Lamport**.

Lògica implementada: L'algorisme assegura que les peticions s'atenen segons l'ordre en què s'han emès lògicament ("First Come, First Served" distribuït).

1. Estat: S'incorpora una variable Clock a l'estat del procés, inicialitzada a 0.

2. Sincronització i Actualització:

- En enviar (request): Abans de demanar el bloqueig, s'incrementa el rellotge local ($\text{Clock} = \text{Clock} + 1$) i s'envia aquesta marca de temps (timestamp) juntament amb la petició ($\{\text{Clock}, \text{MyId}\}$).
- En rebre (request): Quan arriba un missatge, s'actualitza el rellotge local per respectar la causalitat: $\text{NewClock} = \max(\text{LocalClock}, \text{RemoteClock}) + 1$.
- **Regla de Decisió:** Dins de l'estat d'espera, la prioritat es determina comparant la marca de temps de la nostra petició amb la rebuda. Té prioritat qui tingui el rellotge més baix (més antic). En el cas excepcional d'empat en els rellotges, s'utilitza l'ID del node per desempatar de manera determinista.

Codi destacat:

None

```
wait(Nodes, Master, Refs, Waiting, TakeRef, MyId, Clock, MyRequestTS) ->
    receive
        {request, From, Ref, RemoteClock, RemoteId} ->
            % Actualizamos reloj local
            NewClock = erlang:max(Clock, RemoteClock) + 1,

            % Comparamos Timestamps: {RemoteClock, RemoteId} vs {MyClock,
MyId}
            % Si el remoto es menor (ocurrio antes), tiene prioridad.
            {MyPrioClock, _} = MyRequestTS,

            PriorityRemote = (RemoteClock < MyPrioClock) or
                            ((RemoteClock == MyPrioClock) and (RemoteId <
MyId)),

            if
                PriorityRemote ->
                    From ! {ok, Ref},
                    wait(Nodes, Master, Refs, Waiting, TakeRef, MyId,
NewClock, MyRequestTS);
                true ->
                    wait(Nodes, Master, Refs, [{From, Ref}|Waiting],
TakeRef, MyId, NewClock, MyRequestTS)
            end;

            {ok, Ref} ->
                NewRefs = lists:delete(Ref, Refs),
                wait(Nodes, Master, NewRefs, Waiting, TakeRef, MyId, Clock,
MyRequestTS);
            release ->
                ok(Waiting),
                open(Nodes, MyId, Clock)
    end.
```

3. Testing

3.1. Preparació (Compilació)

```
Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
1> c(lock1).
{ok,lock1}
2> c(lock2).
{ok,lock2}
3> c(lock3).
{ok,lock3}
4> c(worker).
worker.erl:10:18: Warning: erlang:now/0 is deprecated; see th
% 10|      {A1,A2,A3} = now(),
%   |                      ^

worker.erl:11:5: Warning: random:seed/3 is deprecated; use th
% 11|      random:seed(A1, A2, A3),
%   |      ^

worker.erl:18:17: Warning: random:uniform/1 is deprecated; us
% 18|      Sleptime = random:uniform(Sleep),
%   |      ^

worker.erl:30:32: Warning: random:uniform/1 is deprecated; us
% 30|      Worktime = random:uniform(Work),
%   |      ^

worker.erl:46:8: Warning: erlang:now/0 is deprecated; see the
% 46|      T1 = now(),
%   |      ^

worker.erl:52:16: Warning: erlang:now/0 is deprecated; see th
% 52|      T2 = now(),
%   |      ^

{ok,worker}
5> c(gui).
{ok,gui}
6> c(muty).
{ok,muty}
```

3.2. Aixecar els Nodes

Obrim 5 terminals diferents. En cadascuna , executem un node amb un nombre específic. I un terminal l'assignem com el master, el que executarà.

- Terminal 1: erl -sname n1

```
C:\Users\dpere\OneDrive\Documents\Ingenieria informática\5o semestre\SODX\LAB2\muty-source-files\muty-source-files>erl -sname n1
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:12:12] [ds:12:12:10] [async-threads:1] [jit:ns]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(n1@David)1> |
```

- Terminal 2: erl -sname n2

```
C:\Users\dpere\OneDrive\Documents\Ingenieria informática\5o semestre\SODX\LAB2\muty-source-files\muty-source-files>erl -sname n2
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:12:12] [ds:12:12:10] [async-threads:1] [jit:ns]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(n2@David)1> |
```

- Terminal 3: erl -sname n3

```
C:\Users\dpere\OneDrive\Documents\Ingenieria informática\5o semestre\SODX\LAB2\muty-source-files\muty-source-files>erl -sname n3
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:12:12] [ds:12:12:10] [async-threads:1] [jit:ns]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(n3@David)1> |
```

- Terminal 4: erl -sname n4

```
C:\Users\dpere\OneDrive\Documents\Ingenieria informática\5o semestre\SODX\LAB2\muty-source-files\muty-source-files>erl -sname n4
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:12:12] [ds:12:12:10] [async-threads:1] [jit:ns]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(n4@David)1> |
```

- Terminal 5: erl -sname master

```
C:\Users\dpere\OneDrive\Documents\Ingenieria informática\5o semestre\SODX\LAB2\muty-source-files\muty-source-files>erl -sname master
Erlang/OTP 28 [erts-16.1] [source] [64-bit] [smp:12:12] [ds:12:12:10] [async-threads:1] [jit:ns]

Eshell V16.1 (press Ctrl+G to abort, type help(). for help)
(master@David)1> |
```

3.3. Executar la prova

Al terminal 5 (master), compilem tots els codis (ex: "c(muty).") i els carreguem per a que totes les terminals ho tinguin (ex: "nl(muty).")).

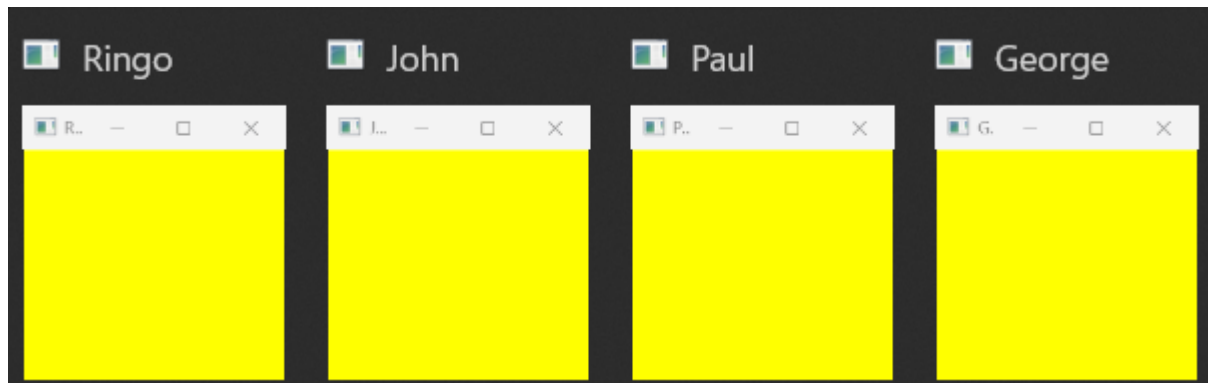
I també abans d'executar assignem una variable Nodos, amb els nodes de cada terminal, en aquest cas es "@David" ja que així els assigna:

- Nodos = ['n1@David', 'n2@David', 'n3@David', 'n4@David'].

I després, ja podem executar tots els locks i comprovar el seu funcionament.

3.3.1 Test Lock1

Podem veure missatges de “giving up”.



```
(master@David)35> muty:start(lock1, 100, 100, Nodos).
ok
Paul: lock taken in 1 ms
Paul: lock released
John: lock taken in 33 ms
John: lock released
George: lock taken in 98 ms
George: lock released
Ringo: lock taken in 129 ms
Ringo: lock released
Paul: lock taken in 177 ms
Paul: lock released
John: lock taken in 179 ms
John: lock released
George: lock taken in 229 ms
George: lock released
Ringo: lock taken in 230 ms
Ringo: lock released
Paul: lock taken in 262 ms
Paul: lock released
John: lock taken in 198 ms
John: lock released
George: lock taken in 227 ms
George: lock released
Ringo: lock taken in 132 ms
Ringo: lock released
Paul: lock taken in 246 ms
Paul: lock released
John: giving up
George: giving up
```

```
John: 14 locks taken, 660.2857142857143 ms (avg) for taking, 3 withdrawals
Ringo: 14 locks taken, 673.9285714285714 ms (avg) for taking, 3 withdrawals
Paul: 13 locks taken, 710.7692307692307 ms (avg) for taking, 3 withdrawals
George: 13 locks taken, 699.6153846153846 ms (avg) for taking, 3 withdrawals
```

3.3.2 Test Lock2

Ja no hi ha missatges de "giving up". Però, tot i així, hi ha un error d'exclusió mútua (Col·lisions). Si ens fixem en les línies de la captura:

- Línia 7: John: lock taken in 763 ms (en John entra).
- Línia 8: Paul: lock taken in 1049 ms (en George entra).
- Línia 9: John: lock released (en John surt).

Un problema molt greu! Entre la línia 7 i la 9, en John tenia el bloqueig (estava a dins). No obstant això, a la línia 8, en Paul també hi ha entrat. Això significa que dos processos tenien el bloqueig alhora.

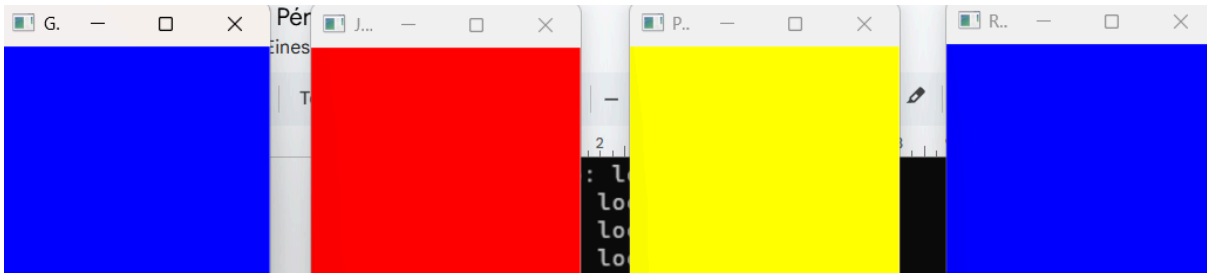


```
(master@David)37> muty:start(lock2, 1000, 1000, Nodos).
ok
Paul: lock taken in 2 ms
Paul: lock released
George: lock taken in 125 ms
George: lock released
Ringo: lock taken in 119 ms
Ringo: lock released
John: lock taken in 763 ms
Paul: lock taken in 1049 ms
John: lock released
Paul: lock released
George: lock taken in 844 ms
George: lock released
Paul: lock taken in 166 ms
Paul: lock released
Ringo: lock taken in 259 ms
John: lock taken in 196 ms
John: lock released
Ringo: lock released
George: lock taken in 243 ms
George: lock released
Paul: lock taken in 260 ms
Paul: lock released
John: lock taken in 820 ms
John: lock released
Ringo: lock taken in 1045 ms
```

```
Ringo: 63 locks taken, 520.5238095238095 ms (avg) for taking, 0 withdrawals
John: 70 locks taken, 333.7142857142857 ms (avg) for taking, 0 withdrawals
Paul: 58 locks taken, 633.8448275862069 ms (avg) for taking, 0 withdrawals
George: 51 locks taken, 804.9411764705883 ms (avg) for taking, 0 withdrawals
```


3.3.3 Test Lock3

Ara amb els rellotges de lamport ja no veiem ni missatges de “giving up” ni l’error d’exclusió mútua que es veia en el “lock2”.



```
(master@David)39> muty:start(lock3, 1000, 1000, Nodos).
ok
Ringo: lock taken in 5 ms
Ringo: lock released
George: lock taken in 600 ms
George: lock released
John: lock taken in 1029 ms
John: lock released
Paul: lock taken in 1317 ms
Paul: lock released
Ringo: lock taken in 852 ms
Ringo: lock released
John: lock taken in 955 ms
John: lock released
George: lock taken in 1111 ms
George: lock released
Paul: lock taken in 1113 ms
Paul: lock released
John: lock taken in 761 ms
John: lock released
Ringo: lock taken in 1243 ms
Ringo: lock released
Paul: lock taken in 1018 ms
Paul: lock released
George: lock taken in 1034 ms
George: lock released
John: lock taken in 1076 ms
```

```
Ringo: 33 locks taken, 1137.878787878788 ms (avg) for taking, 0 withdrawals
John: 34 locks taken, 991.6764705882352 ms (avg) for taking, 0 withdrawals
George: 34 locks taken, 1139.7941176470588 ms (avg) for taking, 0 withdrawals
Paul: 33 locks taken, 1145.6969696969697 ms (avg) for taking, 0 withdrawals
```

4. Respostes a les Preguntes Obertes

A continuació es responen les preguntes plantejades a la pràctica.

Pregunta 1: What is the behavior of the lock when you increase the risk of a conflict? (Quin és el comportament del bloqueig quan augmenta el risc de conflicte?)

Resposta: Quan augmenta la concurrència (el risc de conflicte), el sistema basat en lock1 entra en un estat d'interbloqueig (deadlock). Això succeeix perquè l'algorisme és ingenu: un procés espera rebre l'OK de tots els altres per entrar. Si dos o més processos intenten accedir al recurs simultàniament, tots passen a l'estat d'espera (waiting) i deixen de respondre a les peticions dels altres. Com a resultat, es forma un cicle d'espera infinit on ningú cedeix el pas. Això s'evidencia en els logs amb els missatges continus de giving up, que indiquen que el temps d'espera (withdrawal) s'ha esgotat sense èxit.

Pregunta 2.1: Justify how your code guarantees that only one process is in the critical section at any time. (Justifica com el teu codi garanteix que només un procés està a la secció crítica en qualsevol moment.)

Resposta: El codi intenta garantir l'exclusió mútua assignant una prioritat estàtica única a cada node basada en el seu identificador (on l'ID 1 té la màxima prioritat). La garantia es basa en la regla següent: si dos nodes competeixen, el node amb ID més alt (menor prioritat) està obligat a enviar un missatge ok i cedir el pas al node amb ID més baix, quedant-se ell a l'espera. Nota: Tot i aquesta lògica teòrica, a la pràctica hem observat que aquesta garantia és feble en un entorn asíncron, ja que poden produir-se condicions de carrera si els missatges es creuen, provocant que dos processos entrin temporalment a la vegada (col·lisió), tal com advertia l'enunciat.

Pregunta 2.2): What is the main drawback of lock2 implementation? (Quin és el principal inconvenient de la implementació lock2?)

Resposta: El principal inconvenient és la manca de justícia (unfairness). En dependre d'una prioritat fixa, els nodes amb ID baix (com el node 1) sempre guanyen la competició per accedir al recurs. Això pot provocar situacions d'inanició (starvation) per als nodes amb ID alt (com el node 4), que podrien veure's incapaços d'accedir a la secció crítica si el sistema té una càrrega de treball elevada.

Pregunta 3: Note that the workers are not involved in the Lamport clock. According to this, would it be possible that a worker is given access to a critical section prior to another worker that issued a request to its lock instance before (assuming real-time order)? (Tingues en compte que els workers no participen en el rellotge de Lamport. Segons això, seria possible que un worker accedeixi a la secció crítica abans que un altre que hagi fet la petició abans (assumint ordre en temps real)?)

Resposta: Sí, és possible. Els rellotges de Lamport ordenen els esdeveniments basant-se en la causalitat (la relació causa-efecte produïda per l'intercanvi de missatges), i no en el temps físic real. Si dos workers demanen accés al recurs de manera concurrent (és a dir, sense haver-se comunicat entre ells prèviament), els seus rellotges lògics locals podrien no estar sincronitzats amb el pas del temps real. Per exemple, el Worker A podria demanar el

lock a les 10:00:01 (temps real) amb un rellotge lògic de 5, i el Worker B demanar-ho a les 10:00:00 (abans en temps real) però amb un rellotge lògic de 10. L'algorisme de Lamport donarà prioritat al Worker A perquè $5 < 10$, tot i que en temps real hagi estat posterior. Això passa perquè, sense intercanvi de missatges, el sistema no té manera de saber quin esdeveniment ha passat "abans" realment.